

```
#!/bin/bash
```

```
# Función para generar una cadena aleatoria de letras mayúsculas o minúsculas
```

```
#function generar_cadena_letras {
```

```
generar_cadena_letras(){
```

```
    local longitud="$1"
```

```
    local letras_mayusculas="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
    local letras_minusculas="abcdefghijklmnopqrstuvwxyz"
```

```
    local cadena=""
```

```
    # local: variable local que solo existe dentro de la función
```

```
    for (( i=0; i<longitud; i++ )); do
```

```
        local tipo_letra=$((RANDOM % 2))
```

```
        # RANDOM: [0-32767]
```

```
        # % 2 se utiliza para calcular el módulo 2 del número aleatorio,
```

```
        # lo que significa que el resultado estará en el rango de 0 a 1
```

```
        if [ $tipo_letra -eq 0 ]; then
```

```
            letra="{letras_mayusculas:$((RANDOM % ${#letras_mayusculas})):1}"
```

```
            # $((RANDOM % ${#letras_mayusculas})): ${#letras_mayusculas}
```

```
            # devuelve la longitud de la cadena almacenada en
```

```
            # letras_mayusculas, que es el número total de letras
```

```
            # mayúsculas disponibles. RANDOM % ${#letras_mayusculas} genera
```

```
            # un número aleatorio entre 0 y la longitud de la cadena de
```

```
            # letras mayúsculas.
```

```
            # ${letras_mayusculas:$((RANDOM % ${#letras_mayusculas})):1}":
```

```
            # Esta es una expansión de parámetros que extrae una subcadena
```

```
            # de letras_mayusculas
```

```
            # La sintaxis ${variable:start:length} se utiliza para extraer
```

```
            # una subcadena de la variable variable que comienza en la
```

```
            # posición start y tiene una longitud length
```

```
        else
```

```
            letra="{letras_minusculas:$((RANDOM % ${#letras_minusculas})):1}"
```

```
        fi
```

```
        cadena+="$letra"
```

```
    done
```

```
    echo "$cadena"
```

```
}
```

```
# Función para generar una cadena aleatoria de números
```

```
#function generar_cadena_numeros
```

```
generar_cadena_numeros() {
```

```
    local longitud="$1"
```

```
    local cadena=""
```

```
    for (( i=0; i<longitud; i++ )); do
```

```
        numero=$((RANDOM % 10)) #[0-9]
```

```
        cadena+="$numero"
```

```
    done
```

```
    echo "$cadena"
```

```
}
```

Función para generar una cadena aleatoria de letras mayúsculas, minúsculas y números

#function generar_cadena_alfanumerica

generar_cadena_alfanumerica(){

local longitud="\$1"

local cadena=""

for ((i=0; i<longitud; i++)); do

tipo_caracter=\$((RANDOM % 3))

case \$tipo_caracter in

0) letra=\$(echo {a..z} | tr -d ' ' | cut -c\$((RANDOM % 26 + 1))) ;;

1) letra=\$(echo {A..Z} | tr -d ' ' | cut -c\$((RANDOM % 26 + 1))) ;;

2) letra=\$((RANDOM % 10)) ;;

echo {a..z}: Genera una secuencia de letras minúsculas del

alfabeto, de la 'a' a la 'z'.

tr -d ' ': Elimina cualquier espacio en blanco que pueda

estar presente en la secuencia de letras.

cut -c\$((RANDOM % 26 + 1)): Corta la secuencia de letras en

una posición aleatoria

esac

case \$variable in

0)

valor_1)

*): default

esac: fin del switch case

cadena+="\$letra"

done

echo "\$cadena"

}

Comprobamos si se proporcionaron los argumentos correctos

if [\$# -lt 1] || [\$# -gt 2]; then

echo "Uso: \$0 <longitud de la cadena> [tipo de cadena]"

exit 1

fi

Asignamos los valores de los argumentos

longitud="\$1"

tipo_cadena="\$2"

Si no se proporcionó el tipo de cadena, lo pedimos al usuario

if [-z "\$tipo_cadena"]; then

read -p "Introduce el tipo de cadena (alfanum, alfa o num): " tipo_cadena

fi

Verificamos si el tipo de cadena es válido

case \$tipo_cadena in

alfanum|alfa|num) ;;

alfanum|alfa|num): son los patrones que se van a comparar

Si \$tipo_cadena coincide con alguno de estos, entonces

se ejecutará el código entre ;; ;;

*) echo "Tipo de cadena no válido. Debe ser alfanum, alfa o num."

exit 1

;;

esac

```
# Generamos la cadena según el tipo especificado
case $tipo_cadena in
  alfa) cadena=$(generar_cadena_letras "$longitud") ;;
  num) cadena=$(generar_cadena_numeros "$longitud") ;;
  alfanum) cadena=$(generar_cadena_alfanumerica "$longitud") ;;
  # llamada_a_la_función $variable
  # ;; -> para indicar fin del case
esac

# Imprimimos la cadena generada
echo "$cadena"
```